Computational Economics

Lab: Working with AI Agents

Yaolang Zhong yaolang.zhong@e.u-tokyo.ac.jp
Graduate School of Economics, The University of Tokyo

Lab Overview

- **Large language models**
  - Model families and developers
  - Performance, reliability, cost, and capabilities
- **The harness**
  - Tools and execution interfaces: CLI and MCP
  - Requests, results, context, and permissions
- **The agent**
  - LLM + harness and agent applications
  - Coordinating a task from input to checked output

LLM Families and Developers

A **large language model (LLM)** processes context and generates text, code, or requests to perform actions.

- **GPT family** — developed by **OpenAI**.
  Examples: GPT-6 Astra, GPT-6.1 Sol, GPT-6 Luna.
- **Claude family** — developed by **Anthropic**.
  Examples: Claude Fable 5, Claude Opus 5.5, Claude Sonnet 5.5.
- **Gemini family** — developed by **Google DeepMind**.
  Examples: Gemini 4 Argon, Gemini 3.8 Flash.
- **Grok family** — developed by **xAI**.
  Examples: Grok 4.20, Grok 4.7.
- **DeepSeek family** — developed by **DeepSeek**.
  Examples: DeepSeek-V4-Pro, DeepSeek-V4.1-Flash.

How LLMs Are Evaluated

LLMs are commonly judged along the following dimensions.

- **Task performance**: accuracy or success on one attempt (**pass@1**). For example, Terminal-Bench tests tasks performed with terminal tools.
- **Reliability**: factual errors, instruction-following failures, and consistency across repeated runs.
- **Token use and cost**: a **token** is a unit of text, often part of a word. Cost depends on token usage (including reported reasoning) and token prices; a useful measure is **cost per completed task**.
- **Speed**: time to the first answer token, output tokens per second, and total completion time. Longer reasoning can increase total time.
- **Input capabilities**: supported languages and **multimodal inputs**, such as text, images, audio, and video.
- **Long-context performance**: context-window size and accuracy when using long documents.

Scores depend on the test setup: extra attempts, additional tools, or more reasoning time can improve results while increasing cost.

The Harness: Tools and Interfaces

A **harness** is the software surrounding an LLM. It supplies tools, manages context and permissions, and returns tool results to the LLM.

- **Files and coding:** search, read, create, and edit files.
- **Execution:** run Python, terminal commands, and tests.
- **Web access:** search the web, retrieve pages, and operate a browser.
- **External services:** access a database, cloud drive, or other connected service.

Tools can be accessed through different interfaces:

- A **command-line interface (CLI)** operates software through text commands. The harness can run `git status` to inspect repository changes.
- **Model Context Protocol (MCP)** standardizes how applications connect to servers providing tools and information.

How the Harness Accesses Tools

The harness executes requests and supplies results as input to the next LLM call.

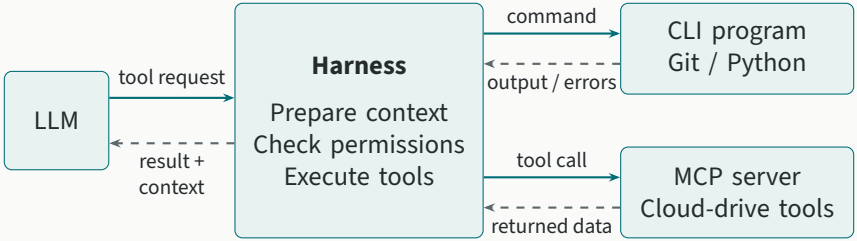

Figure 1: Solid arrows: requests. Dashed arrows: returned information.

- The **command runner** and **MCP client** are components used by the harness.
- CLI and MCP can work together in one task; an MCP server can itself run a CLI program. These are examples of tool access, not an exhaustive list.

The Agent: Coordinating a Task

An **LLM-based AI agent** pursues a task by selecting actions, using tools, and adapting its next actions to the results.

LLM + harness = AI agent

Agent applications: Codex (OpenAI), Claude Code (Anthropic), Cursor Agent (Cursor), Gemini CLI (Google), and GitHub Copilot (GitHub).

Example: a weekly temperature chart

You have: a shared-drive `data.csv` with `date` and `temperature_c`.

You want: a checked chart in date order, with Celsius units, and a reusable Python script.

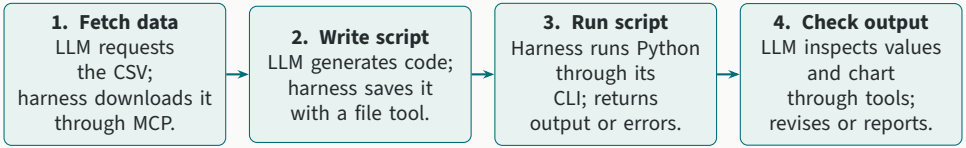

Figure 2: Read left to right: each step uses the previous step's result.

References and Further Reading

1. **OpenAI, Agent and Codex documentation. [Documentation]**
   - Codex as a platform: build on the open agent harness
2. **Anthropic, Claude model and Claude Code documentation. [Documentation]**
   - Claude Code: Overview
3. **Microsoft, Visual Studio Code: agent and terminal documentation. [Documentation]**
   - Understand agent harnesses
4. **Model Context Protocol contributors, Model Context Protocol documentation. [Documentation]**
   - Architecture overview
5. **Artificial Analysis, Benchmarking methodology. [Web resource]**